

Roleta (curvas), o paradoxo da rotação da moeda e o paradoxo da roda de Aristóteles

Osvaldo L. Santos-Pereira * ^{†1}

¹Universidade Federal do Rio de Janeiro, Rio de Janeiro, RJ, Brasil

2 de janeiro de 2026

Resumo

Este trabalho discute o conceito de roleta, as curvas geradas que ocorrem quando uma curva rola sem deslizar ao longo de outra, traçando o caminho de um ponto fixo. O paradoxo da rotação da moeda e o paradoxo da roda de Aristóteles são utilizados como motivações pedagógicas para discutir as equações paramétricas de epicicloides e hipocicloides, fornecendo uma intuição geométrica para as demonstrações matemáticas e a implementação computacional dessas curvas. Códigos em Python foram apresentados como motivação para aplicar as equações paramétricas deduzidas, resultando em visualizações e animações.

1 Introdução

Roletes são geradas quando uma curva rola sem deslizar ao longo de outra, traçando o caminho de um ponto fixo. Exemplos clássicos incluem cicloides, epicicloides, hipocicloides e cardioides [1, 2, 3, 4, 5, 6, 7]. Apesar de sua construção simples, as roletes frequentemente produzem resultados surpreendentes e contraintuitivos.

Dois casos famosos ilustram como interpretações erradas podem surgir da análise das curvas de roleta; exemplos são o paradoxo da rotação da moeda e o paradoxo da roda de Aristóteles. Uma epicicloide consiste em um círculo externo rolando ao redor de outro círculo fixo, e o paradoxo da moeda foi inicialmente definido como um caso particular de uma epicicloide: uma moeda rolando ao redor de outra moeda de raio igual completa duas rotações, e não uma, como a intuição sugere

*olsp@if.ufrj.br

[†]ORCID: 0000-0003-2231-517X

[8, 9, 10]. No paradoxo da roda de Aristóteles, rodas concêntricas de diferentes raios parecem percorrer a mesma distância, aparentemente contradizendo a relação entre circunferência e raio [11, 12, 13].

O paradoxo da moeda possui um análogo na mecânica celeste: o conceito de tempo sideral. Um dia sideral [14] representa o tempo de uma rotação completa em torno do eixo do planeta em relação às estrelas distantes, de forma análoga ao paradoxo da rotação da moeda. Um ano terrestre possui aproximadamente 365,25 dias solares, mas 366,25 dias siderais. Enquanto um dia solar tem 24 horas, um dia sideral possui aproximadamente 23 horas, 56 minutos e 4,1 segundos.

Uma epicicloide é uma curva geométrica traçada por um ponto em um círculo que rola sem deslizar ao redor do exterior de um círculo fixo. O paradoxo da rotação da moeda é ilustrado como um caso particular da curva epicicloide, traçada por um ponto em uma moeda rolando sobre outra moeda fixa de mesmo tamanho. A curva apresenta pontos em que entra em contato com o círculo fixo, e o número de cúspides N é igual à razão entre o raio R do círculo fixo e o raio r do círculo rolante, de modo que $N = R/r$ quando essa razão é um número inteiro.

Abad [15] apresentou uma aplicação interessante das roletes, na qual calculou o campo magnético gerado por uma corrente estacionária que assume a forma de dois tipos de curvas especiais: hipocicloides e epicicloides com um número especificado de lados. O cálculo foi realizado no centro das curvas. Ele utilizou a lei de Biot-Savart, e o resultado mostrou-se geral, pois é obtido como uma função do número de lados da curva e em termos de um parâmetro que identifica o tipo de curva considerada.

Este artigo apresenta uma introdução pedagógica às curvas de rolete, destacando sua formulação matemática, interpretação física e potencial educativo. é dada atenção especial às epicicloides, hipocicloides e aos paradoxos do movimento de rolamento: o paradoxo da rotação da moeda e o paradoxo da roda de Aristóteles. Uma das intenções deste trabalho é demonstrar como esses paradoxos se conectam com os aspectos intuitivos da geometria e da mecânica no contexto do ensino de física.

Este trabalho está organizado da seguinte forma: a segunda seção apresenta e discute o paradoxo da rotação da moeda, resolvendo-o por meio de cinemática circular, que pode ser aprendida em livros-texto de física básica [16]. A terceira seção apresenta as curvas epicicloides, deduzindo as equações paramétricas da curva e fornecendo diversos exemplos dessas curvas. A quarta seção introduz a hipocicloide, em que é feita uma simples mudança de sinais em r , como uma analogia à dedução das equações paramétricas das epicicloides. São fornecidos vários exemplos de curvas para diferentes valores de k . A quinta seção apresenta e

explica o paradoxo da roda de Aristóteles no contexto da física fundamental [16]. A sexta seção apresenta todos os códigos Python utilizados neste trabalho para gerar as imagens e animações. A última seção traz a conclusão deste artigo.

2 Paradoxo da rotação da moeda

O paradoxo da rotação da moeda foi originalmente proposto por Martin Gardner em 1975 [8]. Gardner foi um matemático conhecido por escrever enigmas matemáticos recreativos. Esse aparente paradoxo ganhou notoriedade em 1982, quando foi proposto como uma questão em um exame do SAT. O SAT é um teste padronizado amplamente utilizado para admissão em faculdades nos Estados Unidos. O problema era que não havia resposta correta entre as cinco alternativas. Alguém percebeu que a questão não podia ser respondida corretamente e relatou o erro à empresa responsável pela aplicação do SAT, o que gerou uma grande repercussão amplamente noticiada em jornais de prestígio nos Estados Unidos ¹.

O paradoxo da moeda foi novamente abordado em 2015 na plataforma de streaming YouTube, por um canal chamado *MindYourDecisions*, em um vídeo intitulado “Why did everyone miss this SAT Math question?” ², publicado em 5 de julho de 2015. Em outubro de 2025, o vídeo já contava com mais de três milhões de visualizações.

Um artigo intitulado “The SAT Problem That Everybody Got Wrong” foi escrito por Jack Murtagh, editado por Jeanna Bryner e publicado na revista *Scientific American* em 20 de junho de 2023 ³. O artigo abordava o problema do SAT de 1982 e o paradoxo da rotação da moeda.

Em 30 de novembro de 2023, o canal do YouTube *Veritasium* também publicou um vídeo sobre o paradoxo da rotação da moeda e a polêmica sobre a questão do SAT. Em outubro de 2025, o vídeo do Veritasium intitulado “The SAT Question Everyone Got Wrong” ⁴ já contava com mais de dezesseis milhões de visualizações. Um fato curioso é que o Dr. Doug Jungreis, atualmente pesquisador em matemática e uma das pessoas que relataram o erro na questão do SAT em 1982, apareceu no vídeo e foi entrevistado pelo Dr. Derek Muller, proprietário do canal Veritasium.

O paradoxo da moeda é o seguinte: coloque duas moedas idênticas sobre uma mesa, ambas com a cara voltada para cima e alinhadas. Mantenha a moeda B

¹<https://www.nytimes.com/1982/05/25/us/error-found-in-sat-question.html>

²<https://www.youtube.com/watch?v=kN3AOMrnEUs>

³<https://www.scientificamerican.com/article/the-sat-problem-that-everybody-got-wrong/>

⁴<https://www.youtube.com/watch?v=FUHkTs-Ipfg>

fixa e faça a moeda A rolar ao seu redor sem deslizar. Quando a moeda A alcança o lado oposto, as caras estão novamente paralelas, e a moeda A completou uma rotação. Ao continuar o movimento de volta ao ponto de partida, ocorre uma segunda rotação. Paradoxalmente, a moeda A parece ter rolado uma distância duas vezes maior que sua circunferência.

A Fig. 1 abaixo ilustra uma das razões pelas quais esse paradoxo parece surgir no raciocínio das pessoas. A imagem mostra cinco instantes da moeda rolante sobre uma moeda fixa: o primeiro na posição inicial, com a cara voltada para cima; o segundo instante, após a moeda rolante ter virado sua cara para baixo; o terceiro instante mostra a moeda rolante após completar metade do percurso circular, novamente com a cara voltada para cima; o quarto instante mostra a moeda novamente com a cara voltada para baixo, mas à esquerda da moeda fixa; e o último instante mostra a moeda rolante retornando ao ponto de partida. Em geral, as pessoas contam o número de vezes que o topo da moeda (cara) rolante toca a moeda fixa, imaginando que isso corresponde à metade do rolamento total.



Figura 1: Ilustração do paradoxo da rotação da moeda: uma moeda rolando externamente ao redor de outra moeda idêntica completa duas rotações antes de retornar à sua posição inicial. A imagem mostra moedas brasileiras de um real (1 BRL).

A explicação é que a circunferência da moeda A é igual à distância rolada, mas uma rotação adicional surge porque seu trajeto é circular e não retilíneo. Ao “achatar” a circunferência da moeda B em uma linha, observa-se apenas uma rotação real, enquanto a segunda corresponde ao transporte da moeda A ao redor da moeda B.

O centro da moeda rolante segue uma trajetória circular de raio igual à soma dos dois raios, de modo que sua trajetória possui o dobro da circunferência de cada moeda. Percorrer essa distância sem deslizar obriga a moeda rolante a completar duas rotações. Detalhes sobre a orientação da moeda ou o sentido de rotação não afetam esse resultado.

Esse paradoxo é facilmente explicado utilizando a cinemática simples do movimento circular. Considere um círculo A de raio r rolando sem deslizar sobre o exterior de um círculo fixo B de raio R , como mostrado na Fig. 2; o centro do círculo A descreve uma trajetória circular de raio $R + r$.

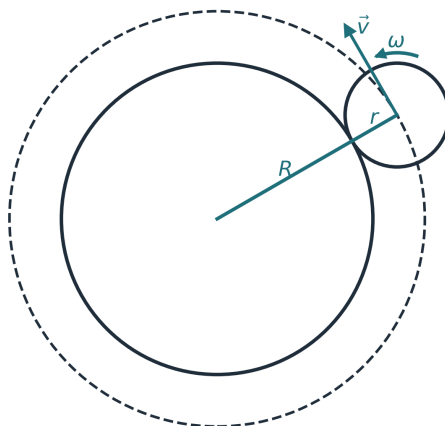


Figura 2: geometria de um círculo de raio r rolando sem deslizar ao redor de um círculo fixo de raio R . O centro do círculo móvel descreve uma trajetória circular de raio $R + r$, resultando em $(R + r)/r$ rotações completas em torno do seu próprio eixo.

Seja a velocidade linear do centro de A igual a v , e sua velocidade angular em torno de seu próprio centro igual a ω . A condição de não deslizamento no ponto de contato entre os dois círculos implica que a velocidade linear do centro do círculo A é

$$v = \omega r. \quad (1)$$

O tempo necessário para que o centro de A complete uma revolução completa ao redor de B é

$$t = \frac{2\pi(R + r)}{v} = \frac{2\pi}{\omega} \left(\frac{R}{r} + 1 \right). \quad (2)$$

Durante esse intervalo, a distância angular $\theta = \omega t$ percorrida pelo círculo A em torno de seu próprio centro é dada pela seguinte expressão

$$\theta = 2\pi \left(\frac{R}{r} + 1 \right). \quad (3)$$

Portanto, o número de rotações completas de A em torno de seu próprio eixo é

$$N = \frac{\theta}{2\pi} = \frac{R}{r} + 1. \quad (4)$$

Como caso particular do paradoxo da moeda, se $R = r$, então

$$N = \frac{r}{r} + 1 = 2, \quad (5)$$

significando que o círculo A completa duas rotações em torno de seu centro ao rolar uma vez ao redor do círculo B .

3 Epicicloide

Uma epicicloide é um tipo de curva rolete gerada por um ponto fixo na circunferência de um círculo de raio r que rola sem deslizar ao redor do exterior de um círculo fixo de raio R , exatamente como o exemplo explorado na Sec. 2 sobre o paradoxo da rotação da moeda. O lugar geométrico do ponto traça uma curva com uma rica variedade de formas, dependendo da razão $k = R/r$. Para $k = 3$, como mostrado na Fig. 3, tem-se uma trefoil [6]. Nesse exemplo, trata-se do caso de um círculo de raio r rolando sem deslizar ao redor de um círculo B de raio $R = 3r$.

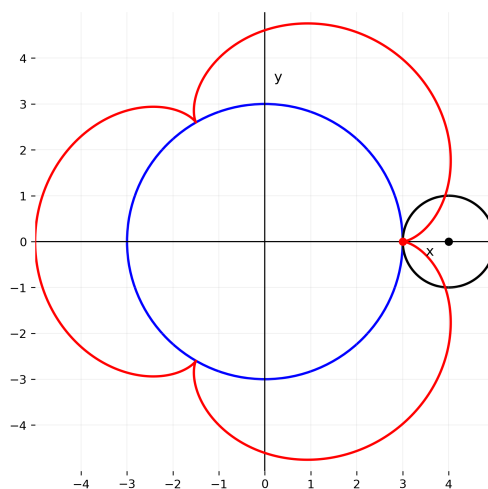


Figura 3: exemplo de uma epicicloide gerada por um círculo de raio r rolando ao redor de um círculo fixo de raio R . O número de cúspides depende da razão R/r . Nesta figura, $R = 3r$, e o número de voltas completadas pelo círculo menor é igual a 4. O nome dessa epicicloide é trefoil [6].

Epicicloides são encontradas em diversos contextos físicos e de engenharia. Elas descrevem o movimento de engrenagens planetárias [17, 18] e curvas cáusticas em

óptica [19], com aplicações em cinemática e em projetos assistidos por computador. Também fornecem exemplos didáticos elegantes de como o movimento de rolamento se traduz em trajetórias geométricas complexas, porém tratáveis.

É simples demonstrar as equações paramétricas da epicicloide analisando a geometria do rolamento sem deslizamento. Considere um círculo de raio r rolando externamente sem deslizar sobre um círculo fixo de raio R , conforme ilustrado na Fig. 4. Considere a origem do sistema de coordenadas no centro do círculo maior, em $O_R(0,0)$. Um ponto $B = (x,y)$ na circunferência do círculo rolante traça a epicicloide.

A Ref. [6] apresenta uma demonstração visual das equações paramétricas de uma rolete geral (*epitrocoide*), traçada por um ponto P fixado em um círculo que rola ao redor de um círculo fixo. Veja as Figs. 56–58 do capítulo 6 da Ref. [6]. A Fig. 4 baseia-se na dedução apresentada em [6]; entretanto, o leitor pode encontrar diversas outras demonstrações na internet, em páginas pessoais de professores, arquivos públicos ou enciclopédias on-line de acesso livre, como a Wikipedia. É comum que livros de cálculo e geometria analítica deixem essa demonstração como exercício para os estudantes. Trata-se de um exercício relativamente trivial, mas deve-se ressaltar que a Fig. 4 não constitui uma demonstração original, uma vez que o estudo das roletes vem desde 1525 (Dürer) [6].

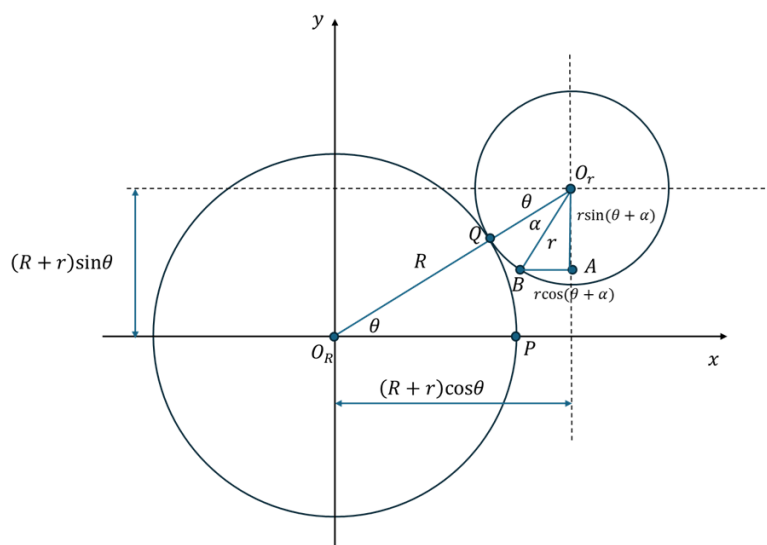


Figura 4: construção da epicicloide. Um círculo de raio r rola externamente ao redor de um círculo fixo de raio R , e um ponto B em sua circunferência traça a curva epicicloide.

Seja θ o ângulo correspondente ao comprimento de arco no círculo fixo, medido a

partir do ponto de tangência inicial $B_0 = (x_0, y_0)$, e seja α o ângulo correspondente ao comprimento de arco no círculo rolante, medido do ponto de contato até o ponto B . Note que, para $\theta = 0$, o ponto inicial está em $B_0 = (R, 0)$.

As coordenadas do ponto B no círculo rolante podem ser derivadas a partir das relações geométricas e trigonométricas mostradas na Fig. 4.

$$x(\theta) = (R + r) \cos \theta - r \cos(\theta + \alpha), \quad (6)$$

$$y(\theta) = (R + r) \sin \theta - r \sin(\theta + \alpha). \quad (7)$$

Como o rolamento ocorre sem deslizamento, os dois comprimentos de arco, tanto do círculo fixo $\widehat{PQ} = \theta R$ quanto o do círculo rolante $\widehat{BQ} = \alpha r$ devem ser iguais. Assim, a condição de não deslizamento fornece a seguinte relação

$$\alpha = \frac{R}{r} \theta. \quad (8)$$

Substituindo a Eq. (8) nas Eqs. (6) e (7), obtêm-se as seguintes fórmulas

$$x(\theta) = (R + r) \cos \theta - r \cos \left(\frac{R + r}{r} \theta \right), \quad (9)$$

$$y(\theta) = (R + r) \sin \theta - r \sin \left(\frac{R + r}{r} \theta \right). \quad (10)$$

Estas são as equações paramétricas da epicloide, expressas em termos do ângulo de rolamento θ . A estrutura da curva, incluindo o número de cúspides, depende diretamente da razão $k = R/r$.

A razão $(R + r)\theta/r$ pode ser interpretada fisicamente como a velocidade angular relativa do círculo rolante em comparação com seu movimento translacional ao redor do círculo fixo. Para cada incremento de 2π em θ , ou seja, uma revolução completa do centro ao redor do círculo fixo, o círculo rolante gira por um ângulo de $2\pi(R + r)/r$. Assim, o círculo rolante completa não uma, mas $(R + r)/r$ rotações completas em torno de seu próprio eixo ao realizar uma única revolução ao redor do círculo fixo. Essa é uma das origens do paradoxo observado com a moeda rolante, pois o movimento de rolamento e o transporte do centro do círculo constituem o movimento combinado de um círculo de raio $R + r$, e não apenas r , resultando em mais rotações do que a intuição poderia sugerir.

A Figura 5 ilustra diversos exemplos de epicloides para diferentes valores da razão $k = R/r$, em que R é o raio do círculo fixo e r é o raio do círculo rolante. Quando k é um número inteiro, a epicloide se fecha com $k + 1$ cúspides, produzindo curvas bem conhecidas como a cardioide ($k = 1$), a nefróide ($k = 2$), a trefóiloide ($k = 3$) e a quatrofóiloide ($k = 4$). Quando k é não inteiro ou racional, a curva torna-se

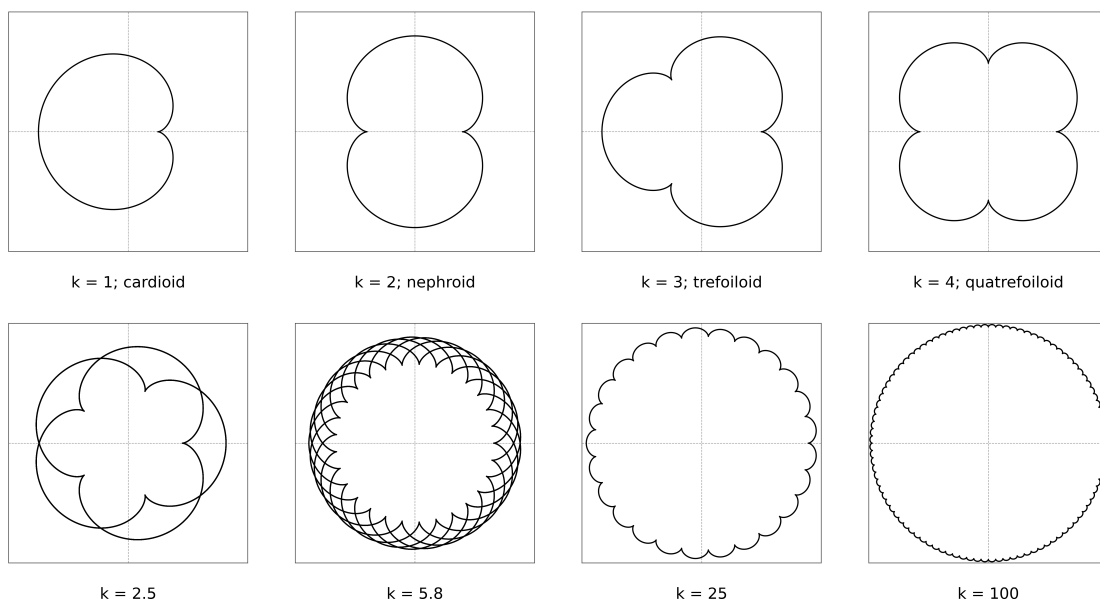


Figura 5: exemplos de epicicloides para diferentes valores de $k = R/r$.

mais intrincada e pode não se fechar após uma única revolução, gerando estruturas densas e semelhantes a rendas.

A Fig. 6 mostra dois pontos de vista para o paradoxo da moeda. A Fig. (a), à esquerda, indica que a moeda menor é desenhada com uma seta que inicialmente aponta para o centro da moeda maior. Do ponto de vista da moeda maior, a moeda menor terá completado uma rotação completa quando a seta voltar a apontar para o centro da moeda maior. Isso ocorre após 120 graus, depois novamente após 240 graus e, finalmente, após uma rotação completa de 360 graus. Sob essa perspectiva, a moeda menor gira três vezes, e cada rotação corresponde à moeda girando ao longo de um terço da circunferência da moeda maior. Portanto, não há paradoxo aqui. No entanto, quando observamos a situação de uma perspectiva externa, vemos uma rotação adicional, pois, ao rolar ao longo da borda da moeda maior, a moeda menor também se move em um grande círculo. A Fig. (b), à direita, mostra que a moeda menor na verdade gira quatro vezes antes de retornar à sua posição inicial. Isso pode parecer muito estranho. Se a moeda maior tem três vezes a circunferência da moeda menor, e a moeda menor rola sem deslizar, como pode levar quatro rotações para voltar ao ponto de partida? Mas é exatamente isso que acontece.

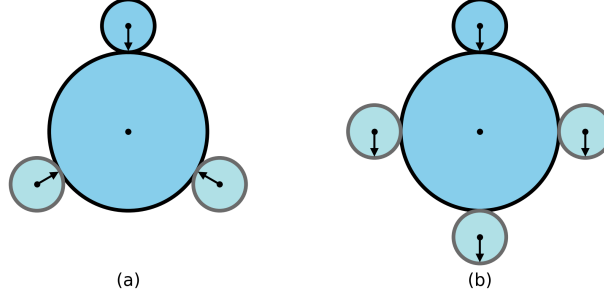


Figura 6: ilustração do paradoxo do rolamento do círculo. **Esquerda:** o raciocínio intuitivo, porém incorreto, em que se assume que o círculo menor gira apenas R/r vezes ao rolar ao redor do círculo maior. **Direita:** o raciocínio correto: a trajetória circular do centro do círculo rolante acrescenta uma rotação extra, resultando em $N = (R + r)/r$.

4 Hipocicloide

Uma hipocicloide é gerada pela trajetória de um ponto fixo em um pequeno círculo que rola dentro de um círculo maior [6]. Isso seria análogo a uma moeda rolando pelo interior de um aro. À medida que o raio do círculo maior aumenta, a hipocicloide se torna mais semelhante à cicloide, que é a curva traçada por um ponto em um círculo ao rolar sobre uma linha reta sem deslizar. A cicloide é uma forma específica de trocoide. A Fig. 7 abaixo ilustra um exemplo de hipocicloide para $k = 3$, denominada deltoide [6].

A dedução das equações paramétricas da hipocicloide é análoga à dedução da epicloide, utilizando uma figura semelhante à da Fig. 4, mas existe uma maneira simples de obter as equações diretamente: basta fazer a substituição $r \rightarrow -r$ para obter as seguintes expressões

$$x(\theta) = (R - r) \cos \theta + r \cos \left(\frac{R - r}{r} \theta \right), \quad (11)$$

$$y(\theta) = (R - r) \sin \theta - r \sin \left(\frac{R - r}{r} \theta \right). \quad (12)$$

Agora o número de rotações completas é reduzido em uma unidade, em vez da adição de uma rotação, à medida que o círculo rolante completa o ciclo de 2π . Uma propriedade interessante da hipocicloide é que, para qualquer valor de r , ela é uma braquistócrona para o potencial gravitacional dentro de uma esfera homogênea de raio R ; veja as páginas 230–232 da Ref. [20].

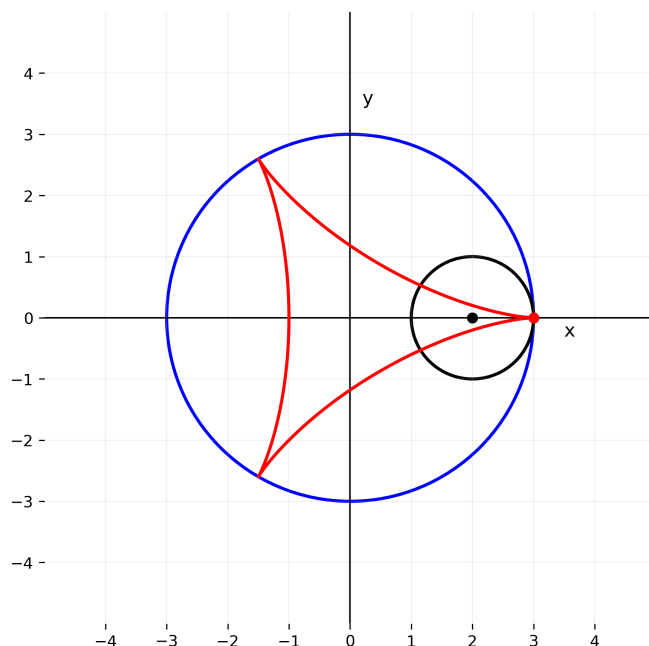


Figura 7: exemplos de hipocicloides para $k = 3$, denominadas deltoides [6].

Como ilustrado na Fig. 8, as hipocicloides exibem uma ampla variedade de formas, dependendo da razão $k = R/r$. Quando k é um número inteiro, a curva é fechada e apresenta exatamente k cúspides: para $k = 3$, obtém-se a deltoide; para $k = 4$, a astroide; e inteiros maiores produzem formas mais complexas em forma de estrela. Se k não é um número inteiro, a curva deixa de se fechar após uma única revolução do círculo rolante. Em vez disso, a trajetória preenche densamente uma região com padrões intrincados semelhantes a estrelas. Essa sensibilidade ao valor de k destaca a riqueza das hipocicloides como exemplos pedagógicos, fornecendo uma maneira natural de conectar geometria, periodicidade e o papel das razões racionais e irracionais na matemática.

5 Paradoxo da roda de Aristóteles

Um exemplo clássico de paradoxo geométrico é conhecido como *paradoxo da roda de Aristóteles*. O arranjo consiste em dois círculos concêntricos de diferentes raios, rigidamente conectados, como se fizessem parte da mesma roda. O círculo maior rola sem deslizar ao longo de uma linha reta.

Intuitivamente, pode-se esperar que, se o círculo maior de raio R percorre uma distância igual à sua circunferência $2\pi R$, então o círculo menor de raio $r < R$ deveria

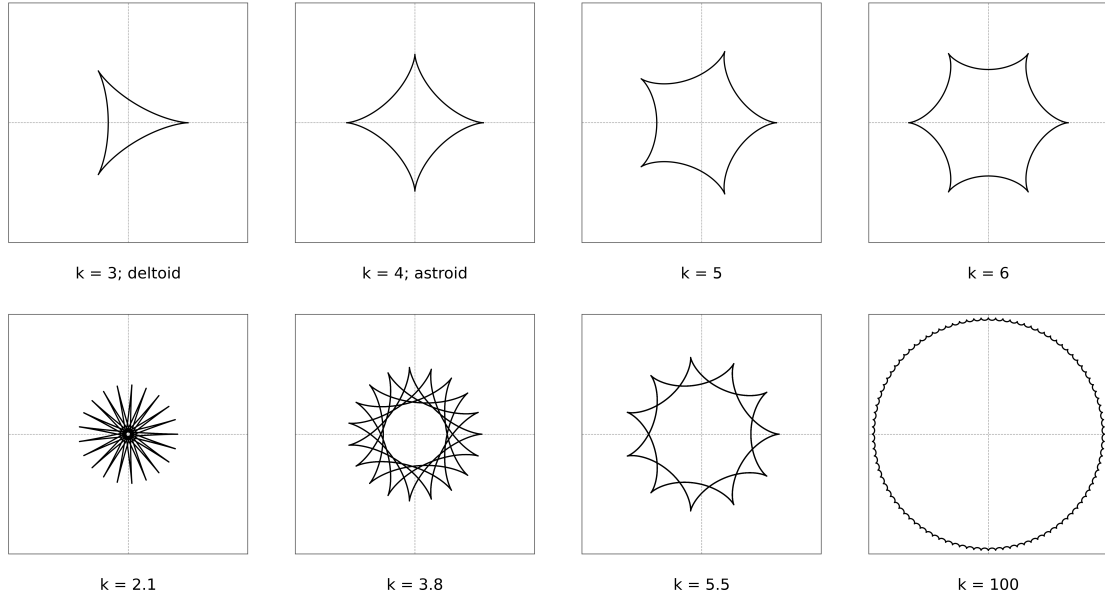


Figura 8: exemplos de hipocicloides para diferentes razões $k = R/r$.

simultaneamente percorrer uma distância igual à sua própria circunferência $2\pi r$. Esse raciocínio parece natural do ponto de vista da cinemática individual de cada círculo. No entanto, como o círculo menor está rigidamente acoplado e gira junto com o círculo maior, ambas as circunferências parecem percorrer a mesma distância linear, apesar de seus diferentes raios e circunferências, levando à conclusão paradoxal de que arcos de tamanhos diferentes seriam iguais em comprimento.

A resolução do paradoxo da roda de Aristóteles está em observar o caminho traçado pelo círculo menor. Embora ele seja concêntrico ao círculo maior, o círculo menor não *rola* sobre o solo. Em vez disso, cada ponto de sua circunferência descreve uma curva que deixa de ser uma linha reta e passa a ser uma *rolete*. Consequentemente, o círculo menor não desenrola de fato toda a sua circunferência sobre o chão. A contradição aparente surge apenas ao assumir, incorretamente, que ambos os círculos rolam da mesma maneira.

A Fig. 9 ilustra o paradoxo da roda de Aristóteles: os círculos antes e depois de uma revolução completa, mostrando os movimentos dos pontos centrais P_R , no círculo de raio R , e P_r , no círculo de raio r , com P_R e P_r iniciando e terminando no topo de seus respectivos círculos. A linha tracejada verde representa o movimento do centro. A curva tracejada azul mostra o movimento de P_R . A curva tracejada vermelha mostra o movimento de P_r . Note que a trajetória de P_r é claramente menor que a de P_R . A razão entre os dois círculos na Fig. 9 é $k = R/r = 2$.

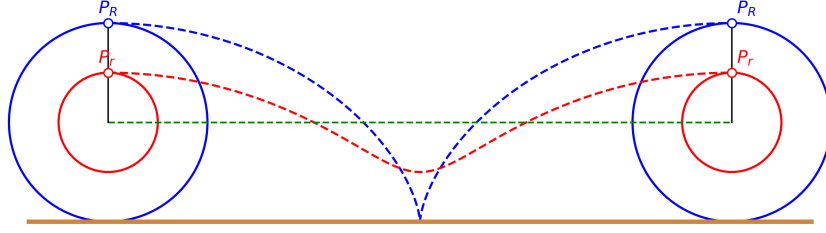


Figura 9: ilustração da solução do paradoxo da roda de Aristóteles para $k = 2$.

Quanto mais próximo P_r estiver do centro, mais curto, direto e próximo da linha verde será o seu trajeto. A Fig. 10 abaixo mostra o caso para a razão $k = 10$.

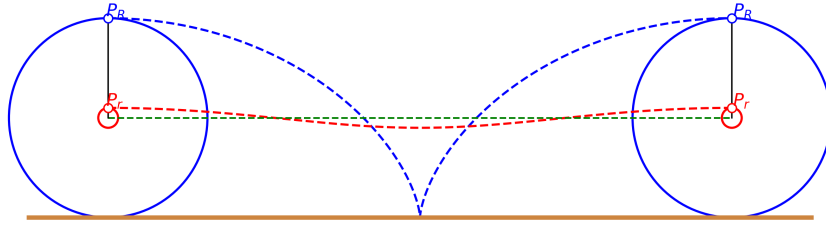


Figura 10: ilustração da solução do paradoxo da roda de Aristóteles para $k = 10$.

As *trocoides* [21, 22, 23] são o nome das curvas descritas pelos pontos P_R no círculo maior de raio R e P_r no círculo menor de raio r , mostradas nas Figs. 9 e 10 acima. A trocoide é uma curva formada quando um círculo rola sem deslizar ao longo de uma linha reta. A restrição cinemática de rolamento sem deslizamento impõe que o deslocamento horizontal (x_c) do centro do círculo seja proporcional ao comprimento de arco percorrido em sua circunferência, $x_c(\theta) = R\theta$, e que a coordenada vertical do centro permaneça constante, $y_c(\theta) = R$, à medida que o círculo rola sobre a linha tomada como eixo x . Se um ponto está localizado a uma distância fixa a do centro, sua posição no referencial de laboratório resulta da superposição da translação do centro do círculo com a rotação do ponto em torno do centro. A representação paramétrica de sua trajetória é, portanto,

$$x(\theta) = R\theta + a \sin \theta \quad (13)$$

$$y(\theta) = R + a \cos \theta. \quad (14)$$

Os aspectos geométricos da curva trocoide dependem da relação entre a distância a e o raio R do círculo. Se $a = R$, a curva traçada reduz-se à *cicloide*, a rolete gerada por um círculo rolando sem deslizar sobre uma linha. Se $a < R$, a trajetória

é uma *trocoide curta* (*curtate trochoid*), caracterizada por arcos que não alcançam a linha de base; enquanto, para $a > R$, obtém-se uma *trocoide longa* (*prolate trochoid*), cujos lóbulos se estendem além da linha de rolamento. A cicloide, como caso particular, desempenhou um papel fundamental na história da matemática e da física, por exemplo, no problema da braquistócrona [24, 25].

6 Códigos Python

6.1 Figura 1: círculos com flechas

Para produzir um proxy de Fig. 1, em vez de moedas, a figura mostra círculos com setas. Execute o código Python fornecido abaixo usando um Jupyter Notebook.

keywordstyle

```

1 import matplotlib.pyplot as plt
2 from matplotlib.patches import Circle, FancyArrowPatch
3
4 def draw_equal_quincunx_with_arrows(radius=1.0, arrow_len=None,
5   savepath=None):
6     """
7     Five equal circles (radius = radius) in quincunx.
8     Outer four are tangent to the center circle (centers at
9     distance d=2*radius).
10    Dashed guide circle goes through the outer centers.
11    Arrows start at the CENTER of each outer circle:
12        top & bottom: upwards; left & right: downwards.
13    The dashed guide is rendered BEHIND the disks (hidden where
14    overlapped).
15    """
16    s = float(radius)
17    if s <= 0:
18        raise ValueError("radius must be positive.")
19
20    d = 2.0 * s
21    if arrow_len is None:
22        arrow_len = 1.1 * s
23
24    centers = {
25        "top": (0.0, d),
26        "right": (d, 0.0),
27        "bottom": (0.0, -d),
28        "left": (-d, 0.0),
29        "center": (0.0, 0.0),
30    }
31
32    # z-order: guide (1) < circles (3) < arrows (4)

```

```

30     z_guide, z_circles, z_arrows = 1, 3, 4
31
32     fig, ax = plt.subplots(figsize=(6, 6))
33     ax.set_aspect('equal')
34     ax.axis('off')
35
36     # --- dashed guide circle FIRST (behind) ---
37     ax.add_patch(Circle((0.0, 0.0), d, ec="#4a5568", fc="none",
38                        lw=1.8, ls="--", zorder=z_guide))
39
40     # --- disks on top of guide (opaque facecolors hide the dashed
41     #       where overlapped) ---
42     for key in ["top", "right", "bottom", "left", "center"]:
43         cx, cy = centers[key]
44         ax.add_patch(Circle((cx, cy), s, ec="#2b6cb0", fc="#e6eefc",
45                             lw=2, zorder=z_circles))
46
47     # --- arrows on top ---
48     def vertical_arrow_from_center(cx, cy, direction):
49         if direction == "up":
50             start, end = (cx, cy), (cx, cy + arrow_len)
51         elif direction == "down":
52             start, end = (cx, cy), (cx, cy - arrow_len)
53         else:
54             raise ValueError("direction must be 'up' or 'down'")
55         a = FancyArrowPatch(
56             start, end, arrowstyle='->', mutation_scale=16,
57             lw=2.0, color="#44515c', zorder=z_arrows
58         )
59         ax.add_patch(a)
60
61     vertical_arrow_from_center(*centers["top"], "up")
62     vertical_arrow_from_center(*centers["bottom"], "up")
63     vertical_arrow_from_center(*centers["left"], "down")
64     vertical_arrow_from_center(*centers["right"], "down")
65
66     # limits include arrows
67     pad = 0.3 * s
68     x_pad = s + pad
69     y_pad = max(s, arrow_len) + pad
70     ax.set_xlim(-d - x_pad, d + x_pad)
71     ax.set_ylim(-d - y_pad, d + y_pad)
72
73     if savepath:
74         fig.savefig(savepath, dpi=300, bbox_inches='tight',
75                     pad_inches=0.1)
76     plt.show()
77     plt.close(fig)

```

```

76
77 # Example:
78 draw_equal_quincunx_with_arrows(radius=1.0, arrow_len=1.1)

```

6.2 Figura 1: círculos com flechas versão animada

Para produzir um proxy da Fig. 1, em vez de moedas, a figura mostra círculos com setas, em estilo de animação. Execute o código Python fornecido abaixo usando um Jupyter Notebook.

```

keywordstyle
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.patches import Circle, FancyArrowPatch
4 from matplotlib.animation import FuncAnimation
5 from IPython.display import HTML
6
7 def animate_two_circles_with_roulette_and_arrow(radius=1.0, frames
      =360, interval=25,
8
8                                     clockwise=False,
8                                     savepath=None,
8                                     fps=30, dpi
8                                     =150):
9
9     R = float(radius); r = R
10    sgn = -1.0 if clockwise else 1.0
11    phi0 = np.pi / 2
12    phi = phi0 + sgn * np.linspace(0.0, 2*np.pi, frames, endpoint
      =False)
13    psi = 2.0 * (phi - phi0)          # external rolling, R=r ->
      factor 2
14    cx = (R + r) * np.cos(phi)
15    cy = (R + r) * np.sin(phi)
16    alpha0 = phi0 + np.pi           # marker initially toward
      origin
17    Px = cx + r * np.cos(psi + alpha0)
18    Py = cy + r * np.sin(psi + alpha0)
19
20    fig, ax = plt.subplots(figsize=(6, 6))
21    ax.set_aspect('equal'); ax.axis('off')
22    center_disk = Circle((0, 0), R, ec="#2b6cb0", fc="#e6eefc",
      lw=2, zorder=3)
23    rolling_disk = Circle((cx[0], cy[0]), r, ec="#d65f5f", fc="#
      fdecec", lw=2, zorder=3)
24    ax.add_patch(center_disk); ax.add_patch(rolling_disk)
25    path_line, = ax.plot([], [], '-', lw=2.0, color='red', zorder
      =2)

```

```

26     P_dot,      = ax.plot(Px[0], Py[0], 'o', ms=6, mfc='white', mec
27                        = '#222', zorder=4)
28     arrow = FancyArrowPatch((cx[0], cy[0]), (Px[0], Py[0]),
29                             arrowstyle='->', mutation_scale=16,
30                             lw=2.0,
31                             color='#44515c', zorder=4)
32
33     ax.add_patch(arrow)
34
35     pad = 0.4 * R
36     ax.set_xlim(-3*R - pad, 3*R + pad)
37     ax.set_ylim(-3*R - pad, 3*R + pad)
38
39     def update(i):
40         rolling_disk.center = (cx[i], cy[i])
41         P_dot.set_data(Px[i], Py[i])
42         path_line.set_data(Px[:i+1], Py[:i+1])
43         ang = psi[i] + alpha0
44         arr_len = 0.9 * R
45         start = (cx[i], cy[i])
46         end = (cx[i] + arr_len*np.cos(ang), cy[i] + arr_len*np.
47               sin(ang))
48         arrow.set_positions(start, end)
49         return rolling_disk, P_dot, path_line, arrow
50
51     anim = FuncAnimation(fig, update, frames=frames, interval=
52                        interval, blit=True)
53
54     # ---- Save if requested ----
55     if savepath is not None:
56         ext = savepath.split(".")[1].lower()
57         if ext in {"mp4", "m4v", "mov"}:
58             from matplotlib.animation import FFMpegWriter
59             anim.save(savepath, writer=FFMpegWriter(fps=fps), dpi=
60                      dpi)
61         elif ext in {"gif"}:
62             from matplotlib.animation import PillowWriter
63             anim.save(savepath, writer=PillowWriter(fps=fps), dpi=
64                      dpi)
65         else:
66             raise ValueError("Unsupported extension; use .mp4 or .
67                               gif")
68
69     plt.close(fig) # close figure to avoid duplicate display in
70                   notebooks
71     return HTML(anim.to_jshtml())
72
73 # Show inline only
74 animate_two_circles_with_roulette_and_arrow(radius=1.0)
75 # Save as MP4 (needs ffmpeg)

```

```

67 animate_two_circles_with_roulette_and_arrow(radius=1.0, savepath="
    roll.mp4", fps=30, dpi=150)
68 # Save as GIF (uses Pillow)
69 animate_two_circles_with_roulette_and_arrow(radius=1.0, savepath="
    roll.gif", fps=20)

```

6.3 Figura 2: solucionando o paradoxo de rotação da moeda com física

Para produzir a Fig. 2, execute o código Python fornecido abaixo usando um Jupyter Notebook, Google Colab ou o ambiente de desenvolvimento de sua preferência.

```

keywordstyle
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.patches import Circle, FancyArrow, Arc
4
5 def draw_rolling_diagram(R=3, r=1, theta_deg=30, save_path=None):
6     """
7     Rolling-circle diagram with:
8     - v drawn from the small-circle center (longer, tangent to
        center path),
9     - w curved arrow placed on TOP of the small circle (outside)
        to avoid crossings,
10    - R and r shown as teal radius segments with midpoint labels
11    """
12    theta = np.deg2rad(theta_deg)
13    RR = R + r
14    O = (0.0, 0.0)
15    C = (RR*np.cos(theta), RR*np.sin(theta))
16
17    fig, ax = plt.subplots(figsize=(6, 6))
18    col, teal = '#1f2d3a', '#1f6f7a'
19
20    # Dashed path of the rolling center
21    ax.add_patch(Circle(O, RR, fill=False, ls='--', lw=1.5, color=
        col))
22
23    # Circles
24    ax.add_patch(Circle(O, R, fill=False, lw=2, color=col))
25    ax.add_patch(Circle(C, r, fill=False, lw=2, color=col))
26
27    # Direction from O to C
28    ang = np.arctan2(C[1]-O[1], C[0]-O[0])
29
30    # Radius R with midpoint label

```

```

31 R_end = (O[0] + R*np.cos(ang), O[1] + R*np.sin(ang))
32 ax.plot([O[0], R_end[0]], [O[1], R_end[1]], color='#1f6f7a',
33         lw=2)
34 R_mid = (O[0] + 0.5*R*np.cos(ang), O[1] + 0.5*R*np.sin(ang))
35 ax.text(R_mid[0], R_mid[1], r"$R$", color='#1f6f7a', fontsize
36         =12, ha='center', va='bottom')
37
38 # Radius r with midpoint label
39 r_end = (C[0] - r*np.cos(ang), C[1] - r*np.sin(ang))
40 ax.plot([C[0], r_end[0]], [C[1], r_end[1]], color='#1f6f7a',
41         lw=2)
42 r_mid = (C[0] - 0.5*r*np.cos(ang), C[1] - 0.5*r*np.sin(ang))
43 ax.text(r_mid[0], r_mid[1], r"$r$", color='#1f6f7a', fontsize
44         =12, ha='center', va='bottom')
45
46 # Velocity vector from small-circle center (tangent)
47 vx, vy = np.cos(ang + np.pi/2), np.sin(ang + np.pi/2)
48 v_len = 1.8
49 ax.add_patch(FancyArrow(C[0], C[1], v_len*vx, v_len*vy,
50                         width=0.025, head_width=0.22,
51                         head_length=0.22,
52                         color='#1f6f7a'))
53 ax.text(C[0] + v_len*vx + 0.12, C[1] + v_len*vy, r"$\vec{v}$",
54         color='#1f6f7a', fontsize=12)
55
56 # w curved arrow OUTSIDE small circle, on top (absolute angles
57 70deg to 110 deg)
58 arc_rad = 1.20 * r
59 start_deg_abs, end_deg_abs = 70, 110
60 arc = Arc(C, 2*arc_rad, 2*arc_rad, theta1=start_deg_abs,
61          theta2=end_deg_abs, color='#1f6f7a', lw=2)
62 ax.add_patch(arc)
63 end_rad = np.deg2rad(end_deg_abs)
64 hx = C[0] + arc_rad*np.cos(end_rad)
65 hy = C[1] + arc_rad*np.sin(end_rad)
66 tdx, tdy = -np.sin(end_rad), np.cos(end_rad) # CCW tangent
67 ax.add_patch(FancyArrow(hx - 0.12*tdx, hy - 0.12*tdy, 0.12*tdx
68                        , 0.12*tdy,
69                        width=0.0, head_width=0.18,
70                        head_length=0.18,
71                        color='#1f6f7a', length_includes_head=
72                        True))
73 ax.text(hx + 0.22, hy + 0.18, r"$\omega$", color='#1f6f7a',
74         fontsize=12)
75
76 # Layout
77 ax.set_aspect('equal', adjustable='box')
78 ax.set_xlim(-RR-1.4, RR+1.8)
79 ax.set_ylim(-RR-1.4, RR+1.8)

```

```

68     ax.axis('off')
69
70     if save_path:
71         plt.savefig(save_path, dpi=300, bbox_inches='tight')
72     return fig, ax
73
74 # Example usage:
75 fig, ax = draw_rolling_diagram(R=3, r=1, theta_deg=30, save_path="
    fig2.png")

```

6.4 Figura 3: epícloide

Para produzir a Fig. 3, é necessário executar o código Python fornecido abaixo.

```

keywordstyle
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.patches import Circle
4
5 def plot_epicycloid(R=3.0, r=1.0, points=2000, save_path=None,
6     show=True):
7     """
8     Plot the epicycloid generated by a circle of radius r rolling
9     around a fixed circle of radius R.
10
11     Parameters
12     -----
13     R : float
14         Radius of the fixed circle.
15     r : float
16         Radius of the rolling circle.
17     points : int
18         Number of sample points for the epicycloid.
19     save_path : str or None
20         If provided, saves the figure to this path.
21     show : bool
22         If True, displays the figure. If False, closes it.
23
24     Returns
25     -----
26     fig, ax : Matplotlib figure and axes objects
27     """
28     k = (R + r) / r
29     theta = np.linspace(0, 2*np.pi, points)
30     x = (R + r) * np.cos(theta) - r * np.cos(k * theta)
31     y = (R + r) * np.sin(theta) - r * np.sin(k * theta)

```

```

32 fig, ax = plt.subplots(figsize=(6,6))
33
34 # Grid
35 ax.set_xlim(-R-r-1, R+r+1)
36 ax.set_ylim(-R-r-1, R+r+1)
37 ax.set_aspect('equal', 'box')
38 ax.set_xticks(np.arange(-R-r, R+r+1, 1))
39 ax.set_yticks(np.arange(-R-r, R+r+1, 1))
40 ax.grid(True, which='both', linestyle='-', linewidth=0.5,
41       alpha=0.25)
42
43 # Axes lines
44 ax.axhline(0, color='k', linewidth=1)
45 ax.axvline(0, color='k', linewidth=1)
46 ax.text(R+r-0.5, -0.3, 'x', fontsize=12)
47 ax.text(0.2, R+r-0.5, 'y', fontsize=12)
48
49 # Fixed circle (blue)
50 ax.add_patch(Circle((0,0), R, fill=False, edgecolor='blue',
51                   linewidth=2))
52
53 # Rolling circle (black) at initial position
54 rolling_center = (R + r, 0.0)
55 ax.add_patch(Circle(rolling_center, r, fill=False, edgecolor='
56 black', linewidth=2))
57
58 # Center dot
59 ax.plot(rolling_center[0], rolling_center[1], 'ko')
60
61 # Epicycloid path
62 ax.plot(x, y, color='red', linewidth=2)
63
64 # Starting point
65 ax.plot([R], [0], 'o', color='red')
66
67 # Clean frame
68 for spine in ax.spines.values():
69     spine.set_visible(False)
70
71 plt.tight_layout()
72
73 if save_path:
74     plt.savefig(save_path, dpi=300, bbox_inches='tight')
75
76 if not show:
77     plt.close(fig)
78
79 return fig, ax

```

```
78 fig, ax = plot_epicycloid(R=3.0, r=1.0, save_path="fig3.png")
```

6.5 Figura 3: versão animada da epicloide

O código Python abaixo produz a Fig. 3 em estilo de animação. Execute o código em seu Jupyter Notebook para visualizar um vídeo em formato MP4 da epicloide sendo traçada.

```
keywordstyle
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.patches import Circle
4 from matplotlib.animation import FuncAnimation
5 from matplotlib import rc
6
7 rc("animation", html="jshtml") # for Jupyter animation display
8
9 def animate_epicycloid(R=3.0, r=1.0, frames=600, interval=20):
10     """
11     Animate the epicycloid generated by a circle of radius r
12     rolling
13     around a fixed circle of radius R.
14
15     Parameters
16     -----
17     R : float
18         Radius of the fixed circle.
19     r : float
20         Radius of the rolling circle.
21     frames : int
22         Number of frames in the animation.
23     interval : int
24         Delay between frames in milliseconds.
25
26     Returns
27     -----
28     ani : matplotlib.animation.FuncAnimation
29         Animation object (renders inline in Jupyter).
30     """
31     k = (R + r) / r
32     theta = np.linspace(0, 2*np.pi, frames)
33     x = (R + r) * np.cos(theta) - r * np.cos(k * theta)
34     y = (R + r) * np.sin(theta) - r * np.sin(k * theta)
35
36     fig, ax = plt.subplots(figsize=(6,6))
37     ax.set_xlim(-R-r-1, R+r+1)
38     ax.set_ylim(-R-r-1, R+r+1)
```

```

38     ax.set_aspect('equal', 'box')
39     ax.set_xticks(np.arange(-R-r, R+r+1, 1))
40     ax.set_yticks(np.arange(-R-r, R+r+1, 1))
41     ax.grid(True, linestyle='-', linewidth=0.5, alpha=0.25)
42     ax.axhline(0, color='k', linewidth=1)
43     ax.axvline(0, color='k', linewidth=1)
44     ax.text(R+r-0.5, -0.3, 'x')
45     ax.text(0.2, R+r-0.5, 'y')
46     for s in ax.spines.values():
47         s.set_visible(False)
48
49     # Fixed circle
50     ax.add_patch(Circle((0,0), R, fill=False, edgecolor='blue',
51         linewidth=2))
52
53     # Rolling circle
54     rolling_circle = Circle((R+r, 0), r, fill=False, edgecolor='
55         black', linewidth=2)
56     ax.add_patch(rolling_circle)
57
58     # Traced point and path
59     trace_dot, = ax.plot([], [], 'ro')
60     trace_line, = ax.plot([], [], 'r-', linewidth=2)
61
62     # Starting point
63     ax.plot([R], [0], 'o', color='red')
64     plt.close(fig)
65
66     def init():
67         rolling_circle.center = (R+r, 0)
68         trace_dot.set_data([], [])
69         trace_line.set_data([], [])
70         return trace_dot, trace_line, rolling_circle
71
72     def update(i):
73         cx = (R + r) * np.cos(theta[i])
74         cy = (R + r) * np.sin(theta[i])
75         rolling_circle.center = (cx, cy)
76         trace_dot.set_data(x[i], y[i])
77         trace_line.set_data(x[:i+1], y[:i+1])
78         return trace_dot, trace_line, rolling_circle
79
80     ani = FuncAnimation(fig, update, init_func=init,
81         frames=frames, interval=interval, blit=
82         True)
83
84     return ani
85
86 ani = animate_epicycloid(R=3.0, r=1.0, frames=600, interval=30)
87 ani # shows animation

```

6.6 Figura 5: diversos exemplos de epicicloides

O código Python abaixo produz a Fig. 5. Execute o código em seu Jupyter Notebook ou no Google Colab Notebook.

```
keywordstyle
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def epicycloid_xy(k, num_points=2400, turns=1.0):
5     r = 1.0
6     R = k * r
7     w = (R + r) / r
8     t = np.linspace(0, 2 * np.pi * turns, num_points)
9     x = (R + r) * np.cos(t) - r * np.cos(w * t)
10    y = (R + r) * np.sin(t) - r * np.sin(w * t)
11    return x, y
12
13 def plot_panel(ax, k, subtitle=None):
14     turns = 1.0 if abs(k - round(k)) < 1e-9 else 12.0
15     x, y = epicycloid_xy(k, num_points=4000, turns=turns)
16     ax.axhline(0, color='0.6', lw=0.8, ls='--')
17     ax.axvline(0, color='0.6', lw=0.8, ls='--')
18     ax.plot(x, y, color='black', lw=1.8)
19     ax.set_aspect('equal', 'box')
20     lim = k + 3.0
21     ax.set_xlim(-lim, lim)
22     ax.set_ylim(-lim, lim)
23     ax.set_xticks([]); ax.set_yticks([])
24     for spine in ax.spines.values():
25         spine.set_linewidth(1.0)
26         spine.set_color('0.3')
27     if subtitle:
28         ax.text(0.5, -0.10, subtitle, transform=ax.transAxes,
29                ha='center', va='top', fontsize=22)
30
31 # Parameters
32 k_list = [1, 2, 3, 4, 2.5, 5.8, 25, 100]
33 subs = ["k = 1; cardioid",
34         "k = 2; nephroid",
35         "k = 3; trefoiloid",
36         "k = 4; quatrefoiloid",
37         "k = 2.5",
38         "k = 5.8",
39         "k = 25",
40         "k = 100"]
41
42 # Make subplot areas bigger
43 fig, axes = plt.subplots(2, 4, figsize=(22, 12)) # Bigger figure
```

```

    = bigger boxes
44 axes = axes.ravel()
45
46 for i in range(len(k_list)):
47     plot_panel(axes[i], k_list[i], subtitle=subs[i])
48
49 # Make panels occupy more space
50 plt.subplots_adjust(left=0.00, right=1, top=1, bottom=0.1,
51                     wspace=0.1, hspace=0.3) # Smaller wspace/
52                                             hspace = bigger boxes
53
54 # Show or save
55 plt.savefig("fig5.png", dpi=300, bbox_inches="tight")
plt.show()

```

6.7 Figura 6: duas visões diferentes sobre o paradoxo da rotação da moeda

Para produzir a Fig. 6, execute o código Python abaixo.

```

keywordstyle
1 import matplotlib.pyplot as plt
2 from matplotlib.patches import Circle, FancyArrow
3 import numpy as np
4
5 def draw_coin(ax, center, radius, facecolor='#87CEEB', edgecolor='
    black',
6               arm_angle=None, arm_len=None, armcolor='black', lw
              =3):
7     """Draw a coin with an optional orientation arrow."""
8     circ = Circle(center, radius, facecolor=facecolor, edgecolor=
9                 edgecolor, lw=lw)
10    ax.add_patch(circ)
11    # center dot
12    ax.plot(center[0], center[1], 'o', color='black', ms=4)
13    # orientation arrow
14    if arm_angle is not None:
15        if arm_len is None:
16            arm_len = radius * 0.9
17        x0, y0 = center
18        x1 = x0 + arm_len * np.cos(arm_angle)
19        y1 = y0 + arm_len * np.sin(arm_angle)
20        arr = FancyArrow(x0, y0, x1 - x0, y1 - y0,
21                          width=0.03*radius, head_width=0.25*radius
22                          , head_length=0.25*radius,
23                          color=armcolor, length_includes_head=True
24                          )

```

```

22         ax.add_patch(arr)
23
24 def tangent_center(big_c, R, r, angle):
25     """Center of a small circle tangent to the big circle at polar
26         angle 'angle'."""
27     return (big_c[0] + (R + r) * np.cos(angle),
28             big_c[1] + (R + r) * np.sin(angle))
29
30 def draw_scene(ax, big_center, R, r, mode='left',
31               big_color='#87CEEB', small_color='#B0E0E6'):
32     """Draw one panel (left or right)."""
33     # Big coin
34     draw_coin(ax, big_center, R, facecolor=big_color, lw=3)
35
36     # Top small coin (tangent at 90) with downward arrow
37     top_c = tangent_center(big_center, R, r, np.pi/2)
38     draw_coin(ax, top_c, r, facecolor=big_color, lw=3, arm_angle=-
39               np.pi/2)
40
41     if mode == 'left':
42         # Two tangent small coins at 210 and 330 with angled
43         # arrows
44         c1 = tangent_center(big_center, R, r, np.deg2rad(210))
45         c2 = tangent_center(big_center, R, r, np.deg2rad(330))
46         draw_coin(ax, c1, r, facecolor=small_color, edgecolor='#6
47             b6b6b',
48                   arm_angle=np.deg2rad(30))
49         draw_coin(ax, c2, r, facecolor=small_color, edgecolor='#6
50             b6b6b',
51                   arm_angle=np.deg2rad(150))
52     else:
53         # Three tangent coins: left (180), right (0), bottom (270)
54         left_c = tangent_center(big_center, R, r, np.pi)
55         right_c = tangent_center(big_center, R, r, 0.0)
56         bottom_c = tangent_center(big_center, R, r, -np.pi/2)
57
58         # Left & right coins: arrows up
59         draw_coin(ax, left_c, r, facecolor=small_color, edgecolor
60                   = '#6b6b6b',
61                   arm_angle=-np.pi/2)
62         draw_coin(ax, right_c, r, facecolor=small_color, edgecolor
63                   = '#6b6b6b',
64                   arm_angle=-np.pi/2)
65
66         # Bottom coin: arrow DOWN (fixed)
67         draw_coin(ax, bottom_c, r, facecolor=small_color,
68                   edgecolor='#6b6b6b',
69                   arm_angle=-np.pi/2)

```

```

63 def print_image(r=0.6, k=3, center_distance=8.0, save_path=None):
64     """Render the two panels and optionally save to file."""
65     R = k * r # enforce  $R = k r$ 
66
67     fig, ax = plt.subplots(figsize=(10, 5))
68     left_center = (-center_distance/2, 0.0)
69     right_center = ( center_distance/2, 0.0)
70
71     draw_scene(ax, left_center, R, r, mode='left')
72     draw_scene(ax, right_center, R, r, mode='right')
73
74     # Labels (a) and (b) under each panel
75     y_label = - (R + r) - 0.8
76     ax.text(left_center[0], y_label, '(a)', fontsize=14, ha='
77         center', va='top')
78     ax.text(right_center[0], y_label, '(b)', fontsize=14, ha='
79         center', va='top')
80
81     # Layout
82     extent = center_distance/2 + (R + r) + 1.0
83     ax.set_xlim(-extent, extent)
84     ax.set_ylim(- (R + r) - 1.5, (R + r) + 1.5)
85     ax.set_aspect('equal', adjustable='box')
86     ax.axis('off')
87
88     if save_path:
89         plt.savefig(save_path, dpi=300, bbox_inches='tight')
90     plt.show()
91
92 # Example
93 print_image(r=0.6, k=3, center_distance=8.0, save_path="fig6.png")

```

6.8 Figura 7: hipocicloide

Para produzir a Fig. 7, execute o código Python abaixo.

```

keywordstyle
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.patches import Circle
4
5 def plot_epicycloid(R=3.0, r=1.0, points=2000, save_path=None,
6     show=True):
7     """
8     Plot the epicycloid generated by a circle of radius r rolling
9     around a fixed circle of radius R.

```

```

10 Parameters
11 -----
12 R : float
13     Radius of the fixed circle.
14 r : float
15     Radius of the rolling circle.
16 points : int
17     Number of sample points for the epicycloid.
18 save_path : str or None
19     If provided, saves the figure to this path.
20 show : bool
21     If True, displays the figure. If False, closes it.
22
23 Returns
24 -----
25 fig, ax : Matplotlib figure and axes objects
26 """
27 k = - (R - r) / r
28 theta = np.linspace(0, 2*np.pi, points)
29 x = (R - r) * np.cos(theta) + r * np.cos(k * theta)
30 y = (R - r) * np.sin(theta) + r * np.sin(k * theta)
31
32 fig, ax = plt.subplots(figsize=(6,6))
33
34 # Grid
35 ax.set_xlim(-R-r-1, R+r+1)
36 ax.set_ylim(-R-r-1, R+r+1)
37 ax.set_aspect('equal', 'box')
38 ax.set_xticks(np.arange(-R-r, R+r+1, 1))
39 ax.set_yticks(np.arange(-R-r, R+r+1, 1))
40 ax.grid(True, which='both', linestyle='--', linewidth=0.5,
41         alpha=0.25)
42
43 # Axes lines
44 ax.axhline(0, color='k', linewidth=1)
45 ax.axvline(0, color='k', linewidth=1)
46 ax.text(R+r-0.5, -0.3, 'x', fontsize=12)
47 ax.text(0.2, R+r-0.5, 'y', fontsize=12)
48
49 # Fixed circle (blue)
50 ax.add_patch(Circle((0,0), R, fill=False, edgecolor='blue',
51                     linewidth=2))
52
53 # Rolling circle (black) at initial position
54 rolling_center = (R - r, 0.0)
55 ax.add_patch(Circle(rolling_center, r, fill=False, edgecolor='
black', linewidth=2))
56
57 # Center dot

```

```

56     ax.plot(rolling_center[0], rolling_center[1], 'ko')
57
58     # Epicycloid path
59     ax.plot(x, y, color='red', linewidth=2)
60
61     # Starting point
62     ax.plot([R], [0], 'o', color='red')
63
64     # Clean frame
65     for spine in ax.spines.values():
66         spine.set_visible(False)
67
68     plt.tight_layout()
69
70     if save_path:
71         plt.savefig(save_path, dpi=300, bbox_inches='tight')
72
73     if not show:
74         plt.close(fig)
75
76     return fig, ax
77
78 fig, ax = plot_epicycloid(R=3.0, r=1.0, save_path="fig7.png")

```

6.9 Figura 7: versão animada da hipocicloide

O código Python abaixo produz a Fig. 7 em estilo de animação. Execute o código em seu Jupyter Notebook para visualizar um vídeo em formato MP4 da epicycloide sendo traçada.

```

keywordstyle
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.patches import Circle
4 from matplotlib.animation import FuncAnimation
5 from matplotlib import rc
6
7 rc("animation", html="jshtml") # for Jupyter animation display
8
9 def animate_epicycloid(R=3.0, r=1.0, frames=600, interval=20):
10     """
11     Animate the epicycloid generated by a circle of radius r
12     rolling
13     around a fixed circle of radius R.
14
15     Parameters
16     -----

```

```

16     R : float
17         Radius of the fixed circle.
18     r : float
19         Radius of the rolling circle.
20     frames : int
21         Number of frames in the animation.
22     interval : int
23         Delay between frames in milliseconds.
24
25     Returns
26     -----
27     ani : matplotlib.animation.FuncAnimation
28         Animation object (renders inline in Jupyter).
29     """
30     k = - (R - r) / r
31     theta = np.linspace(0, 2*np.pi, frames)
32     x = (R - r) * np.cos(theta) + r * np.cos(k * theta)
33     y = (R - r) * np.sin(theta) + r * np.sin(k * theta)
34
35     fig, ax = plt.subplots(figsize=(6,6))
36     ax.set_xlim(-R-r-1, R+r+1)
37     ax.set_ylim(-R-r-1, R+r+1)
38     ax.set_aspect('equal', 'box')
39     ax.set_xticks(np.arange(-R-r, R+r+1, 1))
40     ax.set_yticks(np.arange(-R-r, R+r+1, 1))
41     ax.grid(True, linestyle='-', linewidth=0.5, alpha=0.25)
42     ax.axhline(0, color='k', linewidth=1)
43     ax.axvline(0, color='k', linewidth=1)
44     ax.text(R+r-0.5, -0.3, 'x')
45     ax.text(0.2, R+r-0.5, 'y')
46     for s in ax.spines.values():
47         s.set_visible(False)
48
49     # Fixed circle
50     ax.add_patch(Circle((0,0), R, fill=False, edgecolor='blue',
51         linewidth=2))
52
53     # Rolling circle
54     rolling_circle = Circle((R+r, 0), r, fill=False, edgecolor='
55         black', linewidth=2)
56     ax.add_patch(rolling_circle)
57
58     # Traced point and path
59     trace_dot, = ax.plot([], [], 'ro')
60     trace_line, = ax.plot([], [], 'r-', linewidth=2)
61
62     # Starting point
63     ax.plot([R], [0], 'o', color='red')
64     plt.close(fig)

```

```

63
64     def init():
65         rolling_circle.center = (R+r, 0)
66         trace_dot.set_data([], [])
67         trace_line.set_data([], [])
68         return trace_dot, trace_line, rolling_circle
69
70     def update(i):
71         cx = (R - r) * np.cos(theta[i])
72         cy = (R - r) * np.sin(theta[i])
73         rolling_circle.center = (cx, cy)
74         trace_dot.set_data(x[i], y[i])
75         trace_line.set_data(x[:i+1], y[:i+1])
76         return trace_dot, trace_line, rolling_circle
77
78     ani = FuncAnimation(fig, update, init_func=init,
79                         frames=frames, interval=interval, blit=
80                         True)
81
82     return ani
83
84 ani = animate_epicycloid(R=3.0, r=1.0, frames=600, interval=30)
85 ani # shows animation

```

6.10 Figura 8: diversos exemplos de hipocicloides

Para produzir a Fig. 8, execute o código Python abaixo.

```

keywordstyle
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def epicycloid_xy(k, num_points=2400, turns=1.0):
5     r = 1.0
6     R = k * r
7     w = - (R - r) / r
8     t = np.linspace(0, 2 * np.pi * turns, num_points)
9     x = (R - r) * np.cos(t) + r * np.cos(w * t)
10    y = (R - r) * np.sin(t) + r * np.sin(w * t)
11    return x, y
12
13 def plot_panel(ax, k, subtitle=None):
14     turns = 1.0 if abs(k - round(k)) < 1e-9 else 12.0
15     x, y = epicycloid_xy(k, num_points=4000, turns=turns)
16     ax.axhline(0, color='0.6', lw=0.8, ls='--')
17     ax.axvline(0, color='0.6', lw=0.8, ls='--')
18     ax.plot(x, y, color='black', lw=1.8)
19     ax.set_aspect('equal', 'box')

```

```

20     lim = k + 3.0
21     ax.set_xlim(-lim, lim)
22     ax.set_ylim(-lim, lim)
23     ax.set_xticks([]); ax.set_yticks([])
24     for spine in ax.spines.values():
25         spine.set_linewidth(1.0)
26         spine.set_color('0.3')
27     if subtitle:
28         ax.text(0.5, -0.10, subtitle, transform=ax.transAxes,
29                ha='center', va='top', fontsize=22)
30
31     # Parameters
32     k_list = [3, 4, 5, 6, 2.1, 3.8, 5.5, 100]
33     subs = ["k = 3; deltoid",
34            "k = 4; astroid",
35            "k = 5",
36            "k = 6",
37            "k = 2.1",
38            "k = 3.8",
39            "k = 5.5",
40            "k = 100"]
41
42     # Make subplot areas bigger
43     fig, axes = plt.subplots(2, 4, figsize=(22, 12)) # Bigger figure
44     axes = axes.ravel()
45
46     for i in range(len(k_list)):
47         plot_panel(axes[i], k_list[i], subtitle=subs[i])
48
49     # Make panels occupy more space
50     plt.subplots_adjust(left=0.00, right=1, top=1, bottom=0.1,
51                        wspace=0.1, hspace=0.3) # Smaller wspace/
52                                                hspace = bigger boxes
53
54     # Show or save
55     plt.savefig("fig8.png", dpi=300, bbox_inches="tight")
56     plt.show()

```

6.11 Figura 9: paradoxo da roda de Aristóteles

Para produzir a Fig. 9, execute o código Python abaixo.

keywordstyle

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3

```

```

4 def plot_aristotle_wheel(R=2.0, r=1.0, savepath=None):
5     """
6     Plot Aristotle's Wheel Paradox diagram for two concentric
        rolling circles.
7
8     Parameters
9     -----
10    R : float
11        Radius of the larger circle.
12    r : float
13        Radius of the smaller circle.
14    savepath : str or None
15        If provided, saves the figure to the given path.
16    """
17    theta = np.linspace(0, 2 * np.pi, 600)
18
19    # General trochoid for a point at distance a from center
20    def trochoid(a):
21        x = R * theta + a * np.sin(theta)
22        y = R + a * np.cos(theta)
23        return x, y
24
25    # Trajectories for P_R (a=R) and P_r (a=r)
26    x_R, y_R = trochoid(R)
27    x_r, y_r = trochoid(r)
28
29    # Center line at y = R, spanning exactly 2 pi R
30    x_line = [0.0, 2 * np.pi * R]
31    y_line = [R, R]
32
33    # Figure/axes
34    fig, ax = plt.subplots(figsize=(12, 4))
35    ax.set_aspect('equal')
36    ax.axis('off')
37
38    # Paths
39    ax.plot(x_R, y_R, 'b--', lw=1.5, label="Path of $P_R$")
40    ax.plot(x_r, y_r, 'r--', lw=1.5, label="Path of $P_r$")
41    ax.plot(x_line, y_line, 'g--', lw=1.2, label="Center line")
42
43    # Vertical black lines at start and end (center to top of
        large circle)
44    ax.plot([0, 0], [R, R + R], 'k', lw=1)
45    ax.plot([2 * np.pi * R, 2 * np.pi * R], [R, R + R], 'k', lw=1)
46
47    # Circles at start (x=0) and end (x=2 pi R)
48    left_large = plt.Circle((0.0, R), R, edgecolor='blue', fill=
        False, lw=1.5)
49    left_small = plt.Circle((0.0, R), r, edgecolor='red', fill=

```

```

    False, lw=1.5)
50 right_large = plt.Circle((2 * np.pi * R, R), R, edgecolor='
    blue', fill=False, lw=1.5)
51 right_small = plt.Circle((2 * np.pi * R, R), r, edgecolor='red
    ', fill=False, lw=1.5)
52 for c in (left_large, left_small, right_large, right_small):
53     ax.add_patch(c)
54
55 # Markers and labels: top points on each circle at start/end
56 PR_left = (0.0, R + R)
57 Pr_left = (0.0, R + r)
58 PR_right = (2 * np.pi * R, R + R)
59 Pr_right = (2 * np.pi * R, R + r)
60
61 ax.plot(*PR_left, 'bo', markersize=6, mfc='white')
62 ax.plot(*Pr_left, 'ro', markersize=6, mfc='white')
63 ax.plot(*PR_right, 'bo', markersize=6, mfc='white')
64 ax.plot(*Pr_right, 'ro', markersize=6, mfc='white')
65
66 ax.text(PR_left[0] - 0.2, PR_left[1] + 0.2, r"$P_R$",
67         fontsize=12, color='blue')
68 ax.text(Pr_left[0] - 0.2, Pr_left[1] + 0.2, r"$P_r$",
69         fontsize=12, color='red')
70
71 ax.text(PR_right[0] + 0.1, PR_right[1] + 0.2, r"$P_R$",
72         fontsize=12, color='blue')
73 ax.text(Pr_right[0] + 0.1, Pr_right[1] + 0.2, r"$P_r$",
74         fontsize=12, color='red')
75
76 # Ground line
77 x_pad = max(R, r) * 0.8
78 ax.plot([-x_pad, 2 * np.pi * R + x_pad], [0, 0], color='peru',
79         lw=3)
80
81 # Robust limits: include full circles and labels for any R, r
82 pad_y = max(R, r) * 0.3
83 y_min = min(0.0, R - max(R, r)) - pad_y
84 y_max = R + max(R, r) + pad_y
85 x_min = -R - x_pad
86 x_max = 2 * np.pi * R + R + x_pad
87
88 ax.set_xlim(x_min, x_max)
89 ax.set_ylim(y_min, y_max)
90
91 if savepath:
92     fig.savefig(savepath, dpi=300, bbox_inches='tight',
93                 pad_inches=0.1)
94
95 plt.close(fig) # close to avoid handle leaks
96 return fig

```

```

90
91 # Save as PNG
92 fig = plot_aristotle_wheel(R=2, r=1, savepath="fig9.png")
93 fig

```

6.12 Figura 9: paradoxo da Roda de Aristóteles versão animada

Para produzir a Fig. 9 em estilo de animação, execute o código Python abaixo.

```

keywordstyle
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.patches import Circle
4 from matplotlib.animation import FuncAnimation
5 from IPython.display import HTML
6
7 def animate_aristotle_wheel(R=2.0, r=1.0, frames=300, interval=30,
8     show_trails=True):
9     """
10     Animate Aristotle's Wheel Paradox: two concentric circles of
11     radii R and r
12     roll without slipping over a distance 2piR. Points P_R (a=R)
13     and P_r (a=r)
14     trace trochoids. Returns HTML for inline display in Jupyter.
15     """
16     # Parameter over one full revolution
17     theta = np.linspace(0.0, 2*np.pi, frames)
18
19     # Helpers
20     def trochoid(a, th):
21         x = R*th + a*np.sin(th)
22         y = R + a*np.cos(th)
23         return x, y
24
25     # Precompute paths for trails
26     xR_path, yR_path = trochoid(R, theta) # P_R path
27     xR_path, yR_path = trochoid(r, theta) # P_r path
28
29     # Figure and axes
30     fig, ax = plt.subplots(figsize=(12, 4))
31     ax.set_aspect('equal')
32     ax.axis('off')
33
34     # Static floor and center line
35     x_total = 2*np.pi*R
36     pad_x = max(R, r)*0.8

```

```

34 ax.plot([-pad_x, x_total + pad_x], [0, 0], color='peru', lw=3)
    # floor
35 ax.plot([0, x_total], [R, R], 'g--', lw=1.2)
    # center line y=R
36
37 # Static end markers: vertical lines at start and end
38 ax.plot([0, 0], [R, R+R], 'k', lw=1)
39 ax.plot([x_total, x_total], [R, R+R], 'k', lw=1)
40
41 # Dynamic patches: rolling concentric circles (share same
    center)
42 big = Circle((0, R), R, fill=False, lw=1.8, ec='blue')
43 small = Circle((0, R), r, fill=False, lw=1.8, ec='red')
44 ax.add_patch(big)
45 ax.add_patch(small)
46
47 # Moving points P_R and P_r
48 PR_dot, = ax.plot([], [], 'o', ms=6, mfc='white', mec='blue')
49 Pr_dot, = ax.plot([], [], 'o', ms=6, mfc='white', mec='red')
50
51 # Labels near points
52 PR_lbl = ax.text(0, 0, r"$P_R$", fontsize=12, color='blue',
53                 ha='left', va='bottom')
54 Pr_lbl = ax.text(0, 0, r"$P_r$", fontsize=12, color='red',
55                 ha='left', va='bottom')
56
57 # Optional trails
58 PR_trail, = ax.plot([], [], 'b--', lw=1.2) if show_trails else
    (None,)
59 Pr_trail, = ax.plot([], [], 'r--', lw=1.2) if show_trails else
    (None,)
60
61 # Robust limits (avoid clipping for any R, r)
62 pad_y = max(R, r)*0.35
63 y_min = min(0.0, R - max(R, r)) - pad_y
64 y_max = R + max(R, r) + pad_y
65 x_min = -R - pad_x
66 x_max = x_total + R + pad_x
67 ax.set_xlim(x_min, x_max)
68 ax.set_ylim(y_min, y_max)
69
70 # Animation update
71 def update(i):
72     th = theta[i]
73     # Rolling center (translation only; rotation is encoded in
        trochoid points)
74     center_x = R*th
75     center_y = R
76     big.center = (center_x, center_y)

```

```

77     small.center = (center_x, center_y)
78
79     # Current P_R and P_r
80     xR, yR = trochoid(R, th)
81     xr, yr = trochoid(r, th)
82
83     PR_dot.set_data(xR, yR)
84     Pr_dot.set_data(xr, yr)
85
86     PR_lbl.set_position((xR + 0.08*R, yR + 0.08*R))
87     Pr_lbl.set_position((xr + 0.08*R, yr + 0.08*R))
88
89     # Update trails
90     artists = [big, small, PR_dot, Pr_dot, PR_lbl, Pr_lbl]
91     if show_trails:
92         PR_trail.set_data(xR_path[:i+1], yR_path[:i+1])
93         Pr_trail.set_data(xr_path[:i+1], yr_path[:i+1])
94         artists += [PR_trail, Pr_trail]
95     return artists
96
97     anim = FuncAnimation(fig, update, frames=frames, interval=
98         interval, blit=True)
99     plt.close(fig) # prevent duplicate static output
100     return anim
101
102 # Example (in a Jupyter cell):
103 anim = animate_aristotle_wheel(R=2.0, r=1.0)
104 # Save to MP4 (requires ffmpeg installed)
105 anim.save("aristotle_wheel.mp4", writer="ffmpeg", dpi=150)
106 # Or save to GIF (requires ImageMagick or Pillow support)
107 anim.save("aristotle_wheel.gif", writer="pillow", dpi=100)
108 # Display animation
109 HTML(anim.to_jshtml())

```

7 Conclusão

Este trabalho demonstrou que o paradoxo da moeda, o paradoxo da roda de Aristóteles e a geometria das epicicloides e hipocicloides decorrem diretamente da condição de rolamento sem deslizamento, que leva à contagem de rotações $N = (R \pm r)/r$ para um círculo de raio r rolando sobre ou dentro de um círculo de raio R . A aplicação direta de conceitos básicos de cinemática circular foi utilizada para resolver paradoxos aparentes e estabelecer conexões entre mecânica clássica, geometria e visualização.

As equações paramétricas das epicicloides e hipocicloides foram deduzidas e ilus-

tradas por meio de figuras geradas em Python, fornecendo um conjunto de ferramentas computacionais para o ensino de cinemática circular e geometria de curvas. Como as roletas conectam rotação, movimento restrito com vínculos geométricos e geometria analítica com pouca necessidade de pré-requisitos, elas oferecem uma abordagem visualmente atraente e pedagógica para o ensino de física e matemática em nível de graduação. Como bônus, foi incluído código em Python para figuras animadas.

O uso de recursos visuais, como animações geradas por computador e ferramentas computacionais interativas, pode aprimorar significativamente a pedagogia do ensino de física. Plataformas como o YouTube, com canais como o Veritasium [26] e o 3Blue1Brown [27], tornaram-se recursos comuns para estudantes de graduação e pós-graduação que buscam complementar o ensino universitário tradicional com explicações acessíveis e visualmente envolventes.

8 Declarações do autor

Os autores deste artigo não possuem nenhum conflito a declarar

Referências

- [1] J.C. Maxwell, On the Theory of Rolling Curves. §XXXV in Trans. Roy. Soc. Edin. 16 (1849).
- [2] E.H. Lockwood, *Roulettes. Ch. 17 in A Book of Curves* (Cambridge University Press, Cambridge, England, pp. 138-151, 1967).
- [3] H. Cundy and A. Rollett, *Roulettes and Involutives. §2.6 in Mathematical Models, 3rd ed* (Tarquin Pub., Stradbroke, England, pp. 46-55, 1989).
- [4] M. Gardner, *The Sixth Book of Mathematical Games from Scientific American* (University of Chicago Press, Chicago, IL, p. 128, 1984).
- [5] R.C. Yates, *Roulettes. A Handbook on Curves and Their Properties* (J.W. Edwards, Ann Arbor, MI, pp. 175-185, 1952).
- [6] J.D. Lawrence, *A Catalog of Special Plane Curves* (Dover, New York, NY, 1972).
- [7] D. Zwillinger, *Roulettes (Spirograph Curves). §8.2 in CRC Standard Mathematical Tables and Formulae, 3rd ed* (CRC Press, Boca Raton, FL, 1996).

- [8] M. Gardner, *Penny puzzles. In Mathematical Carnival* (Alfred A. Knopf, New York, chapter 2, pages 12–26, 1975).
- [9] T. Pappas, *The Coin Paradox. The Joy of Mathematics* (World Publ./Tetra, San Carlos, CA, p. 220, 1989.)
- [10] H. Steinhaus, *Mathematical Snapshots, 3rd ed* (Dover, New York, NY, p.145, 1999).
- [11] I.E. Drabkin, *Aristotle's Wheel: Notes on the History of a Paradox* (University of Chicago Press, Chicago, IL, 162–198, 1950).
- [12] D. Ballew, The wheel of Aristotle, Math. Teach. 65 (1972).
- [13] P. Costabel, The Wheel of Aristotle and French Consideration of Galileo's Arguments. Math. Teacher 61 (1968).
- [14] A.K. Bartlett, Solar and Sidereal Time, Popular Astronomy 12 (1904).
- [15] D.R. Abad, The Magnetic field for an n-cusped Epi-and Hypo-Cycloids loop current, Rev. Bras. Ensino Fís. 43 (2021).
- [16] D. Halliday, R. Resnick, and J. Walker, *Fundamentals of Physics Extended, 10th Edition*, Wiley, 2013.
- [17] J. J. Uicker, G. R. Pennock and J. E. Shigley, 2003, *Theory of Machines and Mechanisms*, Oxford University Press, New York.
- [18] B. Paul, 1979, *Kinematics and Dynamics of Planar Machinery*, Prentice Hall.
- [19] J.A. Boyle, arXiv:1406.1736v1 (2014).
- [20] N.C. Rana and P.S. Joag, *Classical Mechanics* (Tata McGraw-Hill, Noida, India, 2001)
- [21] E.A. Whitman, American Mathematical Monthly 50 (1943).
- [22] R.C. Yates, *A Handbook on Curves and Their Properties* (Public Domain Mark 1.0, 1947).
- [23] S. Wagon, *Mathematica in Action* (Springer Science & Business Media, New York, NY, 1999).
- [24] R. Courant and H. Robbins, *What Is Mathematics?: An Elementary Approach to Ideas and Methods, 2nd ed* (Oxford University Press, Oxford, England, 1996).

- [25] S.T. Thornton and J.B. Marion, *Classical Dynamics of Particles and Systems, 5th ed* (Brooks/Cole, Pacific Grove, CA, 2004).
- [26] Veritasium YouTube Channel. <https://www.youtube.com/@veritasium>, accessed in October 15, 2025.
- [27] 3Blue1Brown YouTube Channel. <https://www.youtube.com/c/3blue1brown>, accessed in October 15, 2025.